Learning to Select Context per Code Diff: An Accuracy–Cost Trade-off for LLM Code Review

Michael Liu (msl196) **Ruiding Feng (rf711)** **Li-Hua Peng (lp880)**
Machine Learning II, Rutgers University

Abstract

We study whether selecting context per code change can reduce the input cost of LLM-assisted review while maintaining accuracy. Our planned method trains a small selector for a frozen reviewer. We prepared a repository-based split of 46,518 ContextCRBench examples, completed a Qwen3B QLoRA pilot, and implemented context-policy baselines. The saved pilot outputs reveal malformed base-model answers and an adapter that always predicts the majority class. These findings motivate preprocessing and evaluation changes; they do not establish improved review quality.

Repository: `https://github.com/michaelsliu3/CS536_Project`

1 Research question

Reviewing a change may require its issue description or nearby code, but including every available field increases input cost. We ask: *Can a learned selector choose context per diff while maintaining review accuracy and using fewer tokens than fixed context choices?*

ContextCRBench provides issue, pull request (PR), and surrounding-code context [1]. Repoformer studies selective retrieval for code completion [2]; we investigate context selection for review, focusing on the accuracy–cost trade-off following instructor feedback.

Our target is accuracy within one percentage point of the best fixed strategy with at least 30% fewer extra context tokens, or higher accuracy at a matched budget. The Qwen3B adaptation pilot checks the data and evaluation pipeline and asks whether fine-tuning improves merge-status prediction. It is separate from the main experiment, which will keep Qwen frozen and train the selector.

2 Data and experimental setup

Task and split. The reviewer returns a JSON answer with the Boolean field `is_merged`. Merge status is our benchmark target, but only a proxy for code quality: a merged change can contain a bug, and a rejection can have nontechnical reasons. We exclude explicit outcome fields and human review answers from prompts. Later edits and quoted solutions in issue and PR text still need a chronology audit.

Of 68,938 downloaded JSON records, cleaning removes 22,375 exact normalized-diff duplicates and 45 records from 16 conflicting-label groups, leaving 46,518 examples from 99 repositories. Seed 42 fixes an approximately 80/10/10 repository split, keeping PRs together. Repository sets are disjoint (Table 1); unequal repository sizes explain the example proportions. Validation supports development, while test repositories are reserved. Near-duplicates and temporal leakage remain to be checked.

Go (23.7%) and Python (17.4%) are the largest training languages; C# (39.4%) and Rust (27.8%) dominate validation.

Table 1: Retained data in the saved split. True and False denote merge status.

Split	Examples	Repositories	True	False	True (%)
Train	38,097	79	30,281	7,816	79.5
Validation	3,797	10	3,264	533	86.0
Test	4,624	10	3,336	1,288	72.1

A Python example. Training record `63859.json` comes from pandas PR #53773. The issue concerns a `TypeError` on nested JSON containing `null`. Below are four verbatim diff lines; others are omitted. The + marks additions.

```
+        if data is None:
+            return
         if isinstance(data, dict):
             data = [data]
```

The guard returns on `None`. Its label, `False`, records merge status, not fix correctness. This illustrates an input, not a prediction; the label stays out of the prompt.

Model and adaptation method. The pilot uses Qwen2.5-Coder-3B-Instruct, a decoder-only Transformer with 36 layers, hidden size 2,048, and feed-forward size 11,008. Grouped-query attention has 16 query heads and two key/value heads. It generates JSON autoregressively. Its configuration supports 32,768 positions; our experimental cap is 2,048 tokens.

QLoRA [3] freezes the base weights in 4-bit NF4 with double quantization and bfloat16 computation. We train low-rank adapters on the attention query, key, value, and output projections and the feed-forward gate, up, and down projections. Rank 16, scaling $\alpha = 32$, and dropout 0.05 give 29,933,568 trainable parameters. The saved run uses paged 8-bit AdamW, learning rate 2×10^{-4}, cosine decay, 3% warmup, and weight decay 0.01. Batch size one with eight accumulation steps gives an effective batch size of eight. Training ran for three epochs (14,289 optimizer steps) on one 32-GB RTX 5090; the final adapter is `checkpoint-14289`.

Inputs contain repository/language metadata, the diff, issue and PR text, and code before and after the change. Only JSON-answer and end-of-sequence tokens receive loss; prompt tokens are masked. After right truncation, 63.3% of training and 57.2% of validation sequences reach 2,048 tokens. These post-truncation lengths do not measure discarded tokens.

Evaluation and context policies. Evaluation compares the untouched base, the trained adapter, and a constant prediction chosen from training-label counts on the same saved 200 examples. The two model conditions share prompts, quantization, and deterministic decoding, with at most 2,016 input tokens and 32 generated tokens. Invalid JSON counts as an error and is reported separately. We report accuracy, macro-F1, and per-class recall alongside paired outputs.

For the frozen-reviewer study, diff-only, all-context, fixed-mask, random, and BM25 similarity policies are implemented. They select among issue text, PR text, and code before and after the change; the diff is always present. Assembled prompts are checked against token budgets. We plan to train a compact Transformer selector from reviewer outcomes on training repositories. The selector has not been trained, and context-policy comparisons remain pending. Fixed masks and budgets will be chosen on validation data.

3 Progress and Preliminary Findings

We completed three epochs of QLoRA training with recoverable checkpoints and obtained paired predictions for the same 200 validation IDs. We checked protocol and source hashes and recomputed Table 2. The main challenge is interpreting the outputs: malformed base answers and majority-only adapter predictions prevent a claim of improved review quality. A corrected comparison remains pending.

Table 2: Saved validation pilot under the original strict-JSON protocol ($n = 200$). These diagnostics do not establish improved review quality.

Method	Accuracy (%)	Macro-F1	Invalid (%)
Training-majority constant	87.5	0.467	0.0
Untouched Qwen3B	0.0	0.000	100.0
Qwen3B + trained adapter	87.5	0.467	0.0

4 Lessons Learned and Limitations

Class imbalance. True labels account for 79.5% of training data and 175 of the 200 evaluation examples. The adapter predicts True on every example, matching the majority baseline with 87.5% accuracy but zero recall for the 25 False examples. Its macro-F1 is only 0.467. This is consistent with majority-class bias, but we have not established that imbalance caused the behavior. We will test balanced training sampling or example-weighted answer loss, deriving weights from training labels only. Validation macro-F1 and minority-class recall will guide selection; the original validation and test distributions will be preserved.

Output format and truncation. The base model's zero score has a separate observed explanation: all 200 answers fail the strict parser. The 82 untruncated outputs contain Markdown code fences. CPU tokenization confirms that all 118 truncated inputs lose the assistant-start suffix; their saved outputs are not valid verdicts. Thus, the score combines format failure with task performance and cannot be attributed directly to class imbalance. We will shorten context before rendering the chat, preserve role boundaries and output space, and rerun both models under one newly versioned protocol. Constrained verdict decoding is another candidate to test. We will keep the original strict scores for comparison rather than silently changing the parser.

Merge status remains a proxy for code quality, and chronology and near-duplicate checks are unfinished. Language differences across repositories and only ten validation repositories also limit generalization claims.

5 Next steps

Our immediate task is to validate preprocessing, repeat the pilot comparison, and inspect per-class errors.

We also plan to test more complex predictive models under the same split and corrected evaluation protocol, comparing macro-F1, minority-class recall, and inference cost with Qwen3B.

We will check whether context choices change enough frozen-reviewer judgments to justify a learned selector. If so, training-repository outcomes will supply its targets. We will compare accuracy, macro-F1, extra context tokens, and runtime including selection, with repeated seeds and context-type ablations. Negative findings will also be reported.

Acknowledgments. We thank Professor Lesia Semenova for feedback on the research question and evaluation. We used OpenAI Codex for coding, debugging, and editing assistance. Our team is responsible for checking the implementation, running the experiments, and interpreting the results.

References

[1] R. Hu et al. Benchmarking LLMs for fine-grained code review with enriched context in practice. *arXiv:2511.07017*, 2025. `https://arxiv.org/abs/2511.07017`.

[2] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma. Repoformer: Selective retrieval for repository-level code completion. *ICML*, PMLR 235:53270–53290, 2024.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv:2305.14314*, 2023.